GenResearch-Servidor

Memória, armazenamento, concorrência e cache em um pipeline de recuperação simulada

Universidade Federal de Sergipe - Sistemas Operacionais - AV2 (2026.2)

Discente: Roberdan Tamyr dos Santos Feitosa Junior

Repositório: https://github.com/LeeYooBin/ufs-so-2026-2-av2-genresearch

Vídeo: url

Sumário

Resumo
1. Introdução e pergunta norteadora
2. Ambiente e ferramentas
3. Arquitetura e dados
4. Estratégia experimental
5. Resultados: leitura convencional e mmap (C1–C2)
6. Resultados: concorrência e fila (C3)
7. Resultados: cache compartilhado (C4)
8. Sincronização e condição de corrida
9. Memória virtual, pressão e armazenamento (C5)
10. Logs, robustez e observabilidade complementar
11. Discussão, limitações e conclusão
Referências e artefatos

Resumo

Este trabalho implementa e avalia o GenResearch-Servidor, um pipeline experimental inspirado em recuperação aumentada por geração (RAG), para observar a interação entre memória virtual, armazenamento, filas, threads, cache e sincronização em Linux. A aplicação utiliza corpus sintético, fragmentação de documentos, vetores simulados determinísticos, persistência em JSONL, recuperação lexical, contexto limitado e geração de respostas simuladas. Foram executadas 39 medições distribuídas em 13 configurações, com três repetições por configuração, abrangendo leitura convencional e `mmap`, concorrência, cache, pressão de memória e falhas controladas. Os resultados mostram benefícios dependentes do tamanho do arquivo no uso de `mmap`, grande ganho de vazão quando consultas repetidas cabem no cache e ausência de escalabilidade no cenário com cache protegido por lock global e consultas distintas. A observação por `/proc` confirmou sete threads no processo de serviço e integridade de 80 requisições sob fila limitada. O estudo evidencia que mecanismos corretos de sincronização podem introduzir contenção e que a interpretação de RSS, VSZ e faltas de página exige distinguir memória residente, mapeamentos e cache do sistema operacional.

1. Introdução e pergunta norteadora

Sistemas inspirados em IA generativa e RAG combinam leitura de documentos, construção e consulta de índices, recuperação de contexto, geração de respostas e armazenamento temporário de resultados. Mesmo quando a inferência é simulada, esses componentes disputam recursos administrados pelo sistema operacional: RAM, espaço virtual, cache de páginas, disco e tempo de execução das threads. A pergunta norteadora adotada é: como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e mecanismos de sincronização afetam desempenho, estabilidade e correção de um serviço inspirado em RAG? O objetivo é analisar custos e compromissos de implementação, não avaliar qualidade linguística de modelos de IA.

2. Ambiente e ferramentas

O ambiente experimental foi composto pelo Ubuntu 24.04.3 LTS, executado no Windows Subsystem for Linux 2 (WSL2), com kernel Linux `6.6.87.2-microsoft-standard-WSL2` e Python 3.12.3. O processador utilizado foi um AMD Ryzen 7 7735HS with Radeon Graphics, com oito núcleos físicos e 16 processadores lógicos disponíveis, arquitetura

x86-64 e virtualização fornecida pelo hipervisor Microsoft. A GPU não foi utilizada no processamento experimental.

O sistema disponibilizou aproximadamente 15 GiB de memória RAM e 4 GiB de swap, sem utilização de swap na amostra registrada. Os benchmarks definitivos foram executados no sistema de arquivos ext4 da distribuição Linux, disponibilizado pelo dispositivo virtual `/dev/sdd` e montado na raiz do sistema. No momento da coleta, a utilização do armazenamento era de aproximadamente 2%, com 934,2 GiB disponíveis. Essa escolha evitou que a sobrecarga de acesso ao sistema de arquivos montado do Windows (`/mnt/c`) interferisse na comparação principal.

O projeto foi desenvolvido em Python, utilizando predominantemente módulos da biblioteca padrão, como `threading` , `queue` , `mmap` , `logging` , `json` e `collections` . Não foram utilizados Ollama ou modelos neurais, uma vez que os embeddings e o custo de geração foram simulados, conforme permitido pelo enunciado.

Para a interpretação dos resultados, foram consideradas as diferenças entre os indicadores de memória do Linux. O campo `MemFree` representa a memória imediatamente livre, enquanto `MemAvailable` estima a quantidade de memória utilizável sem pressão significativa, incluindo páginas recuperáveis. O cache de páginas mantém conteúdos de arquivos na RAM, reduzindo a necessidade de novas leituras físicas. Além disso, a memória swap pode armazenar páginas anônimas sob pressão, enquanto faltas de página *minor* são resolvidas sem acesso ao armazenamento e faltas *major* geralmente exigem operações de entrada e saída. Essas distinções são relevantes para a análise do comportamento de índices, embeddings e checkpoints.

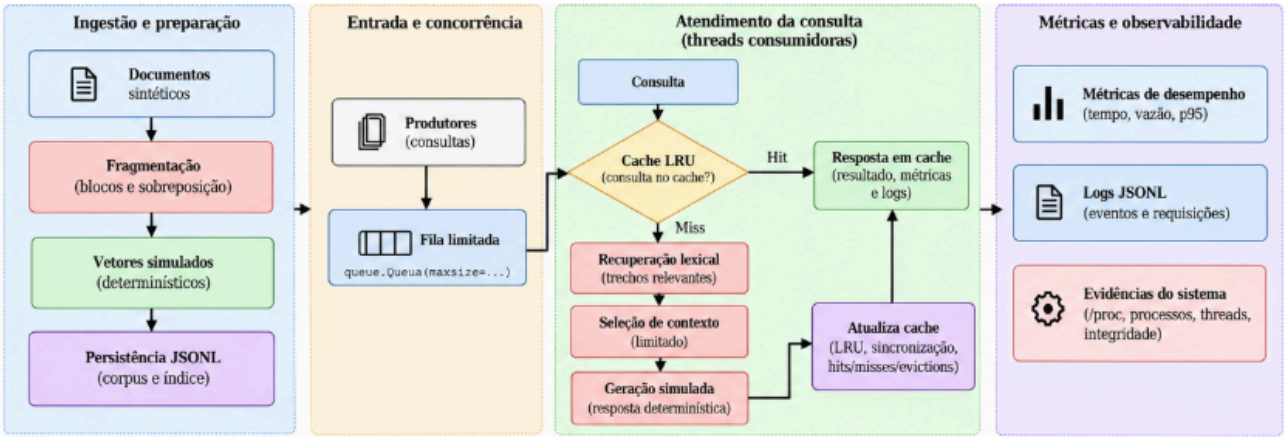
A captura das informações de processos ocorreu em 18/10/2026, aproximadamente às 06h08 UTC, enquanto a coleta complementar de logs da etapa 6.5 foi realizada no mesmo dia, às 21h41 UTC. Como essa coleta complementar ocorreu em `/mnt/c` , seus tempos não foram incorporados às 39 medições definitivas. O commit registrado na evidência do sistema operacional foi `c9cec8b112341aa847a84d5b388b9623b1dc5ac3` .

3. Arquitetura e dados

O fluxo implementado é: documentos sintéticos → fragmentação → vetores simulados → persistência JSONL → recuperação lexical → seleção de contexto limitado → resposta simulada → cache LRU → registro de métricas e logs. Uma fila `queue.Queue(maxsize=...)` coordena produtores e consumidores com bloqueio e sinalização internos. O cache compartilhado mantém resultados recentes e usa sincronização para preservar

consistência. O serviço gera respostas determinísticas resumidas, sem dependência de modelo externo.

Na configuração de 120 documentos foram obtidos 480 fragmentos, com blocos de 40 palavras e sobreposição de oito; os vetores simulados têm 16 dimensões e são determinísticos. A coleta complementar registrou corpus de 119.446 bytes e índice JSONL de 271.569 bytes. A recuperação utiliza correspondência lexical, não busca vetorial por similaridade; os vetores demonstram custos de preparação e persistência, mas não alimentam a ordenação lexical. O contexto é limitado e são registrados quantidade de trechos, bytes e estimativa de tokens.



4. Estratégia experimental

Foram executadas 13 configurações × 3 repetições = 39 medições, mantendo o mesmo programa e variando condições explicitamente definidas. Os fatores incluíram modo de leitura (`open/read` e `mmap`), corpus (120 e 1.200 documentos), número de consumidores (1, 2 e 4), capacidade do cache (2 e 8) e diversidade das consultas (5 ou 40). Os casos foram:

- C1: leitura convencional de corpus pequeno e maior.
- C2: leitura dos mesmos conjuntos via `mmap` .
- C3: dois produtores e 1, 2 ou 4 consumidores, com 40 consultas distintas e cache de capacidade 1.
- C4: 40 consultas com combinações de capacidade do cache e quantidade de consultas distintas.
- C5: retenção de dados com corpus de 120 e 1.200 documentos, sob limites seguros.
- C6: falha controlada de índice ausente e comportamento de fila limitada.

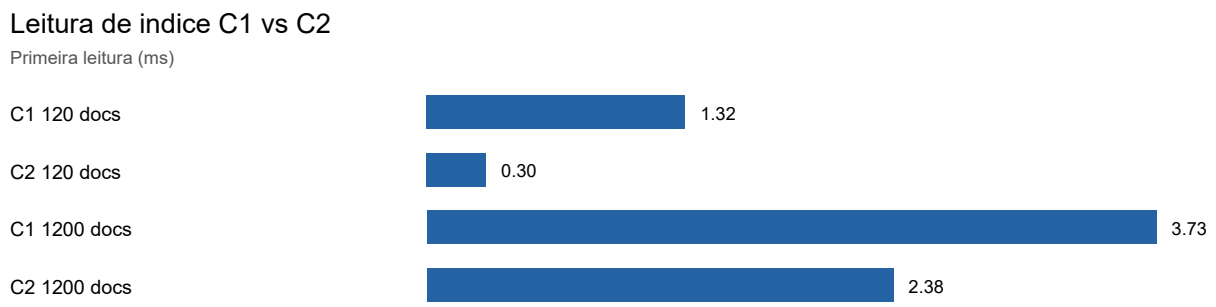
O benchmark registra tempos, vazão, RSS, VSZ, faltas de página, contadores de E/S, integridade e cache. O resumo oficial encontra-se em

`results/metrics/benchmark_summary.csv`, os dados individuais em `results/raw/benchmark_av2.json` e gráficos em `results/plots/`. Cada configuração foi executada em processo separado para reduzir contaminação de estado entre medições. As conclusões são exploratórias: três repetições e conjuntos sintéticos não sustentam generalização estatística ampla.

5. Resultados: leitura convencional e mmap (C1–C2)

Documentos	Convencional, média (ms)	mmap, média (ms)	Redução relativa da média
120	3,383	0,914	73,0%
1.200	8,132	5,242	35,5%

No corpus menor, `mmap` apresentou redução expressiva da média. No corpus maior, o resultado deve ser relativizado: a mediana convencional foi 5,239 ms e a mediana com `mmap`, 5,272 ms; a diferença de médias foi influenciada por uma medição convencional de 14,95 ms. Portanto, não é correto afirmar superioridade consistente de `mmap` em todas as repetições. A implementação lê `mmap[:]`, criando cópia em bytes, e não é zero-copy. O desempenho também depende do cache de páginas e da forma de acesso. Foram registrados tempos de primeira e segunda leitura, mas as amostras não demonstram um ensaio controlado de cache frio.



6. Resultados: concorrência e fila (C3)

Consumidores	Vazão média (req/s)	p95 da latência (ms)
1	37,41	292,11

Consumidores	Vazão média (req/s)	p95 da latência (ms)
2	36,84	345,04
4	36,99	486,27

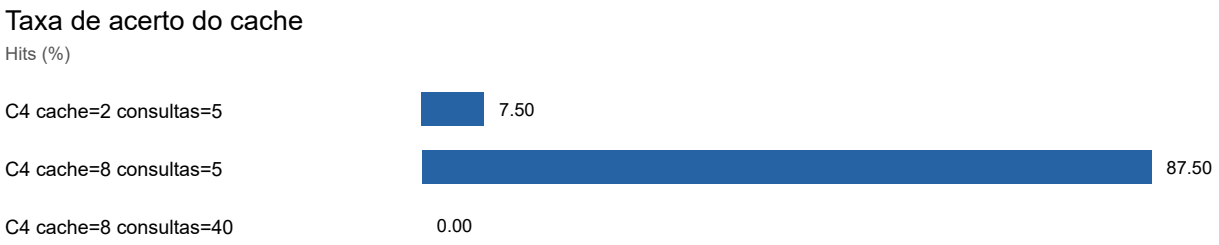
O aumento de consumidores não aumentou a vazão nessa carga; o p95 piorou. A razão mais plausível, apoiada na arquitetura e no experimento de sincronização, é o lock global mantido durante o cálculo de misses no cache. Com 40 consultas distintas e cache de capacidade 1, quase todo trabalho ocorre sob exclusão mútua. Assim, threads adicionais concorrem pelo mesmo recurso em vez de processar respostas em paralelo. O mecanismo escolhido foram threads, adequadas para ilustrar espera, bloqueio e cargas simuladas com `sleep`; o estudo não demonstra aceleração de processamento intensivo em CPU sob o GIL do CPython.



7. Resultados: cache compartilhado (C4)

Cache	Consultas distintas	Hits	Vazão média (req/s)
2	5	7,5%	39,89
8	5	87,5%	290,32
8	40	0%	37,32

Com cinco consultas distintas e cache de oito entradas, a taxa de acerto chegou a 87,5% e a vazão foi aproximadamente 7,28 vezes a da configuração de cache 2. Quando as 40 consultas foram distintas, o cache de oito entradas não obteve hits. Portanto, o benefício depende do padrão de reutilização e da capacidade relativa ao conjunto de trabalho, não apenas de existir cache. A coleta complementar com 40 requisições e cinco consultas distintas registrou 35 hits, cinco misses, zero expulsões e estimativa acumulada de 883,33 ms de trabalho poupado; essa estimativa não deve ser confundida com redução medida do tempo de execução total.



8. Sincronização e condição de corrida

Foi implementada uma demonstração controlada de acesso concorrente ao cache compartilhado, comparando três estratégias: ausência de proteção (*unsafe*), exclusão mútua com lock global e exclusão mútua com lock por chave (*keyed*). O objetivo foi reproduzir uma condição de corrida, verificar sua correção e analisar os custos introduzidos pelos mecanismos de sincronização.

No cenário com oito threads acessando a mesma chave, a implementação insegura realizou oito cálculos redundantes, enquanto as duas variantes sincronizadas realizaram apenas um cálculo e sete reutilizações do resultado. A falha observada foi a duplicação de trabalho, e não a corrupção de respostas. O experimento demonstra que a ausência de sincronização permite que múltiplas threads identifiquem simultaneamente uma entrada ausente e iniciem o mesmo processamento.

No cenário com oito chaves distintas, o lock global apresentou tempo médio de 358,78 ms, enquanto o lock por chave registrou 47,16 ms, uma diferença de aproximadamente 7,61 vezes. Embora o lock global impeça cálculos duplicados, ele também serializa operações independentes. A estratégia por chave mantém a exclusão mútua para consultas iguais, permitindo a sobreposição do processamento de consultas diferentes. Como o custo foi simulado com atrasos, essa diferença representa contenção e concorrência durante períodos de espera, não aceleração de processamento intensivo em CPU.

O serviço principal utiliza um cache LRU compartilhado com lock global. Portanto, os resultados dos experimentos C3 e C4 refletem essa implementação. O lock por chave foi avaliado apenas na demonstração isolada de sincronização, não tendo sido integrado e reavaliado nas 39 execuções definitivas. Assim, a comparação indica uma alternativa promissora para reduzir contenção, mas não comprova ganho equivalente de desempenho no pipeline completo. Os resultados brutos da demonstração estão disponíveis em `results/raw/race_demo.json` .

A fila limitada utiliza `queue.Queue`, que fornece sincronização interna para inserção e retirada de itens, bloqueando produtores quando a fila está cheia e consumidores quando está vazia. Esse comportamento corresponde funcionalmente ao controle de posições livres, posições ocupadas e exclusão mútua do problema clássico de produtores e consumidores.

Na observação do Linux, uma execução com dois produtores, quatro consumidores e fila de capacidade dois registrou 80 requisições submetidas, 80 concluídas, zero perdas, zero duplicações e zero erros. A ocupação máxima atingiu a capacidade configurada. O campo histórico `mean_producer_block_ms` representa a duração total da operação `Queue.put`, não exclusivamente o tempo efetivamente bloqueado, o que limita sua interpretação como medida de contenção.

Em conjunto, os experimentos demonstram que sincronização é necessária para garantir consistência e evitar processamento redundante, mas sua granularidade influencia diretamente a vazão e a latência. A correção de uma condição de corrida, isoladamente, não garante escalabilidade.

9. Memória virtual, pressão e armazenamento (C5)

Corpus	RSS final médio (KiB)	Faltas minor médias	Faltas major
120	20.540	395,7	0
1.200	32.542,7	3.986,3	0

O aumento de corpus elevou o RSS final em cerca de 12.003 KiB, além de elevar as faltas minor. Não houve faltas major registradas nesses cenários, tampouco evidência de uso de swap. Logo, houve maior demanda de memória e atividade de mapeamento/alocação, mas não há base para afirmar thrashing ou paginação intensa em disco. O RSS final do benchmark não representa necessariamente o pico; a evidência complementar de `/proc` fornece `VmHWM` para uma execução específica.

Na observação de processos e threads, o Linux registrou sete threads no PID 6183: uma principal, duas produtoras e quatro consumidoras. Foram observados `VmRSS=19.680 KiB`, `VmSize=469.580 KiB` e `VmHWM=19.840 KiB`. O VSZ elevado não corresponde integralmente à RAM física ocupada: inclui regiões virtuais reservadas e mapeamentos. O ambiente reportou swap total de 4 GiB e uso zero na amostra.

As evidências da observação do sistema operacional estão disponíveis no diretório `docs/evidencias/etapa6_os/`, incluindo informações do ambiente, amostras de processos e threads e verificações de integridade. Esses registros complementam os resultados dos benchmarks, permitindo relacionar métricas coletadas pela aplicação com informações disponibilizadas pelo kernel Linux.

Os contadores de `/proc/[pid]/io` precisam de interpretação: `read_bytes=0` pode ocorrer com leituras atendidas pelo cache de páginas, e taxas aproximadas de syscalls da coleta complementar usam intervalos não exatamente alinhados. A etapa 6.5 observou `write_bytes=401.408`, `syscr=20`, `syscw=181`; esses números não devem ser tratados como benchmark comparável ao C1/C2.

10. Logs, robustez e observabilidade complementar

Uma execução separada, em 18/10/2026, verificou 17 testes automatizados, ingestão de 120 documentos em 19,879 ms, 480 chunks, 40 requisições concluídas de 40 enviadas, fila máxima de dois e média amostral de ocupação após `put` de 1,925. O pipeline alcançou 247,28 req/s com cinco consultas distintas e cache de oito entradas; esse número não integra as 39 medições oficiais e não deve ser comparado diretamente às configurações de carga distintas.

Os logs JSONL registraram `ingestion_completed`, `query_received`, `cache_miss`, `cache_hit`, `retrieval_completed`, `response_generated`, `query_completed`, `pipeline_completed`, `failure` e `shutdown`. O `logging` de Python sincroniza escrita entre threads, mantendo registros completos por linha. A ordem física das linhas não garante ordenação crescente de timestamps quando threads distintas criam registros antes de gravá-los. A falha controlada `FileNotFoundError` foi registrada e seguida de encerramento normal. Não foi implementada política geral de retentativas, pois o recorte experimental priorizou detecção e encerramento controlado.

Os registros completos da execução complementar estão disponíveis em `logs/etapa6_5_events.jsonl`, enquanto as métricas agregadas e os parâmetros utilizados estão em `results/raw/etapa6_5_observability.json`. Esses artefatos permitem verificar a sequência de eventos, a identificação das threads, a integridade das requisições e os indicadores complementares de ingestão, armazenamento, fila e cache.

11. Discussão, limitações e conclusão

Os experimentos demonstram três relações fundamentais entre sistemas operacionais e aplicações inspiradas em recuperação aumentada por geração (RAG): o impacto do armazenamento e da memória virtual sobre a leitura de dados; a influência da sincronização sobre o processamento concorrente; e a dependência do cache em relação ao padrão de reutilização das consultas.

Na comparação entre leitura convencional e `mmap`, foram observadas reduções de tempo médio de aproximadamente 73,0% para 120 documentos e 35,5% para 1.200 documentos. Entretanto, no corpus maior, as medianas foram semelhantes e a média convencional foi influenciada por uma execução mais lenta. Além disso, a implementação utiliza `mmap[:]`, produzindo uma cópia dos dados. Portanto, os resultados indicam vantagens em determinadas condições experimentais, mas não demonstram superioridade universal nem acesso *zero-copy*.

Nos experimentos de concorrência, o aumento de um para quatro consumidores não elevou a vazão, que permaneceu próxima de 37 requisições por segundo, enquanto o p95 da latência aumentou. A arquitetura e a demonstração de sincronização sustentam a interpretação de que o lock global, mantido durante o processamento de consultas não presentes no cache, introduziu serialização e contenção. A comparação isolada com locks por chave mostrou que é possível preservar a correção sem bloquear operações independentes, mas essa alternativa não foi submetida aos benchmarks completos do serviço.

O cache apresentou benefícios expressivos quando o conjunto de trabalho pôde ser armazenado integralmente: com cinco consultas distintas e capacidade de oito entradas, a taxa de acerto atingiu 87,5% e a vazão média foi de 290,32 requisições por segundo. Em contraste, com 40 consultas distintas, a taxa de acerto foi de 0% e a vazão caiu para 37,32 requisições por segundo. Esses resultados demonstram que a eficácia do cache depende da repetição das consultas, da capacidade disponível e da política de substituição.

O aumento controlado do corpus de 120 para 1.200 documentos elevou o RSS final médio de 20.540 KiB para 32.542,7 KiB e aumentou as faltas de página *minor*. Não foram registradas faltas *major* nem evidências de uso de swap nos cenários analisados. Assim, os dados demonstram maior demanda de memória, mas não permitem afirmar ocorrência de *thrashing* ou paginação intensa em armazenamento. O cache de páginas do Linux também influencia a interpretação dos contadores de E/S, pois leituras podem ser atendidas pela RAM sem gerar novas leituras físicas.

As limitações do estudo incluem corpus sintético de tamanho reduzido, embeddings e respostas simulados, recuperação lexical em vez de busca por similaridade vetorial, três repetições por configuração, ausência de controle rigoroso de cache frio, execução sob WSL2 e ausência de inferência real em GPU. Os experimentos também não avaliam checkpoints de modelos com vários gigabytes, múltiplas GPUs, serviços distribuídos ou armazenamento remoto. Por essas razões, os resultados não devem ser generalizados diretamente para sistemas RAG comerciais ou modelos de linguagem de grande porte.

Apesar dessas limitações, o GenResearch-Servidor permitiu observar e relacionar mecanismos concretos de sistemas operacionais, como memória residente e virtual, cache de páginas, faltas de página, filas limitadas, threads, exclusão mútua, contenção, logs concorrentes e tratamento de falhas. Os resultados evidenciam que correção, desempenho e escalabilidade são propriedades distintas: mecanismos de sincronização podem preservar consistência e, simultaneamente, restringir o paralelismo; caches podem reduzir trabalho repetido, mas sua eficácia depende do padrão de acesso.

Conclui-se que a construção de serviços inspirados em RAG exige avaliar não apenas a funcionalidade do pipeline, mas também a interação entre recursos compartilhados e políticas do sistema operacional. A instrumentação, a repetição de experimentos e a análise crítica dos resultados são fundamentais para justificar decisões de arquitetura e reconhecer os limites das otimizações observadas.

Referências e artefatos

- PYTHON SOFTWARE FOUNDATION. *Python 3.12 Documentation*: `threading`, `queue`, `logging`, `mmap` e `collections`. https://docs.python.org/3.12/
- LINUX KERNEL DOCUMENTATION. *The /proc Filesystem*; *Documentation for /proc/[pid]/io*. https://docs.kernel.org/filesystems/proc.html
- Repositório do projeto: https://github.com/LeeYooBin/ufs-so-2026-2-av2-genresearch.
- Vídeo de demonstração: .